

WEEK 4 - Trendleri Takip Etmek Değil, Sinyalleri Okumak

Bu haftaki araştırmaya “2026'nın en popüler teknolojileri neler?” diye başlamadım.

Çünkü popüler olan her teknoloji iş dünyasında karşılık bulmuyor; iş ilanlarında geçen her teknoloji de gerçekten yükselen bir trend olmayabiliyor.

Bu yüzden kendime farklı bir soru sordum:

Eğer iş ilanlarını bir veri seti gibi okusaydım, hangi teknolojiler sürekli tekrar eden pattern'ler olarak karşıma çıkardı?

Araştırmamı da bu soru üzerinden ilerlettim.

1. İş İlanları Bize Ne Söylüyor?

Trend araştırmasına klasik bir “2026'nın en popüler teknolojileri” listesi çıkararak başlamak istemedim. Bunun yerine iş ilanlarını küçük bir veri seti gibi ele aldım.

Türkiye’den ve global pazardan seçtiğim **20 Data odaklı iş ilanında**, ilanda açıkça geçen teknik yetkinlikleri işaretledim. Bir teknoloji ilan metninde yer alıyorsa 1, yer almıyorsa 0 olarak kodladım. Böylece “hangi teknoloji popüler görünüyor?” sorusundan çok, “**şirketler adaylardan gerçekten ne istiyor?**” sorusuna bakmaya çalıştım.

İlk sonuç beklediğim kadar basit değildi.

20 ilanlık örneklemede en sık karşıma çıkan yetkinliklerden bazıları:

- **SQL** → %80
- **Python** → %75
- **Governance / Data Quality** → %75
- **CI/CD** → %65
- **GCP** → %60
- **Data Modeling** → %60
- **BigQuery** → %55

SQL ve Python’ın üst sıralarda olması şaşırtıcı değildi. Benim için asıl dikkat çekici nokta, **Governance / Data Quality’nin Python ile aynı seviyeye gelmesiydi.**

Bu bana şunu düşündürdü:

Data dünyasında artık yalnızca veriyi taşımak veya analiz etmek yeterli değil; üretilen verinin güvenilir, izlenebilir ve yönetilebilir olması da doğrudan bir yetkinlik haline geliyor.

Türkiye ve Global Aynı Şeyi mi Arıyor?

Veriyi Türkiye ve global olarak ayırdığımda farklar daha belirgin hale geldi.

<i>Yetkinlik</i>	<i>Türkiye</i>	<i>Global</i>
<i>SQL</i>	%60	%100
<i>Airflow</i>	%100	%90
<i>dbt / Dataform</i>	%70	%100
<i>Governance / Quality</i>	%60	%90
<i>Python</i>	%70	%80
<i>GCP</i>	%80	%40
<i>Data Modeling</i>	%40	%80

Burada özellikle üç sinyal dikkatimi çekti.

İlki, Airflow.

Türkiye örnekleminde %100, globalde %90 oranında karşıma çıktı. Bu yüzden orchestration konusunu artık “ileride öğrenirim” denecek bir Data Engineering başlığı olarak değil, temel öğrenme planımın parçalarından biri olarak görüyorum.

İkincisi, global ilanlarda dbt/Dataform + Data Modeling + Governance birlikteliği.

Global tarafta yalnızca kod yazabilen değil; veriyi modelleyebilen, transformation sürecini yönetebilen ve veri kalitesini koruyabilen profillerin öne çıktığı görülüyor.

Üçüncüsü ise GCP.

Türkiye örnekleminde %80 iken global örnekleimde %40 çıktı. Ancak bunu “Türkiye’de GCP dünyadan daha yaygın” şeklinde yorumlamak doğru olmaz. Seçtiğim Türkiye ilanlarının bir bölümünün BigQuery/GCP odaklı olması sonucu etkiliyor. Bu yüzden bu çalışma piyasanın tamamını temsil eden bir istatistik değil, **20 ilanlık örneklemden elde edilen bir sinyal analizi.**

Araştırmanın sonunda benim için en önemli çıkarım şu oldu:

İş ilanlarında tek tek teknolojilerden çok teknoloji kümeleri oluşuyor.

Örneğin:

SQL + Python + Data Modeling + dbt/Dataform + Airflow + Governance

aynı veri yaşam döngüsünün farklı parçaları olarak birbirini tamamlıyor.

Bu nedenle öğrenme planımı “hangi aracı öğrenmeliyim?” sorusundan çok:

“Bir veriyi kaynaktan alıp güvenilir bir analitik ürüne dönüştürmek için hangi yetkinlik zincirine ihtiyacım var?”

sorusu üzerinden oluşturmanın daha anlamlı olduğuna karar verdim.

2. BigQuery ile Trend Analizi

Trend araştırmasını yalnızca teoride bırakmak istemedim. Bu yüzden Google Cloud üzerinde **BigQuery Sandbox** kullanarak gerçek bir public dataset üzerinde küçük bir sorgu optimizasyon deneyi yaptım.

İlk olarak NYC Citi Bike verisini kullanarak istasyon bazında yolculuk sayılarını analiz ettim.

İlk sorguda istasyon adıyla birlikte konum bilgilerini de kullandım. BigQuery sorguyu çalıştırmadan önce yaklaşık **1.15 GB** veri işleyeceğini gösterdi.

Daha sonra aynı analizi yalnızca gerçekten ihtiyacım olan sütunlarla tekrar yazdım.

Bu kez işlenen veri miktarı:

1.15 GB → 667.06 MB

seviyesine düştü.

Buradaki fark bana basit ama önemli bir noktayı gösterdi:

BigQuery gibi columnar sistemlerde gereksiz sütunları seçmek yalnızca kodu kalabalıklaştırmıyor; doğrudan işlenen veri miktarını da artırıyor.

LIMIT gerçekten maliyeti düşürüyor mu?

Sonraki adımda başka bir varsayımı test ettim.

Sorguya:

`LIMIT 100`

eklediğimde BigQuery yine:

667.06 MB

işleyeceğini gösterdi.

`LIMIT 100` satırını tamamen kaldırdığımda ise değer yine:

667.06 MB

olarak kaldı.

Yani sonuç ekranında daha az satır görmek, sorgunun daha az veri taradığı anlamına gelmiyordu.

Bu deneyden çıkardığım sonuç:

LIMIT çıktı miktarını sınırlar; ancak taranan veri miktarını tek başına azaltmaz.

Bu küçük test, BigQuery'de optimizasyonun yalnızca “daha kısa SQL yazmak” olmadığını; **hangi sütunların okunduğunun ve verinin nasıl modellendiğinin** daha önemli olduğunu gösterdi.

En büyük fark: Ham veri yerine analitik tablo

Deneyin bence en dikkat çekici kısmı bundan sonra ortaya çıktı.

Public dataset üzerinde her seferinde ham veriyi sorgulamak yerine, istasyon bazındaki yolculuk sayılarını önceden hesaplayıp kendi oluşturduğum:

```
week4_lab.station_trip_summary
```

tablosuna kaydettim.

Daha sonra Top 10 istasyonu bu özet tablo üzerinden sorguladım.

Bu kez BigQuery'nin işleyeceği veri:

20.03 KB

oldu.

Karşılaştırma şöyleydi:

Yaklaşım	İşlenen Veri
Daha fazla sütun içeren public dataset sorgusu	1.15 GB
Yalnızca gerekli sütunlarla public dataset	667.06 MB
Önceden oluşturduğum özet tablo	20.03 KB

Yani aynı analitik soruya cevap verirken ham tablo yerine özet tablo kullanmak, bu örnekte işlenen veriyi yaklaşık **33 bin kat** azalttı.

Bana göre bu deney, Data Warehouse mantığını teorik tanımdan daha iyi açıklıyor.

Problem yalnızca:

“Sorguyu nasıl hızlandırırım?”

değil.

Asıl soru:

“Aynı analizi her seferinde ham veriden hesaplamak zorunda mıyım?”

oluyor.

Burada veri modelleme, aggregate tablolar ve analitik katmanların neden kullanıldığını uygulamalı olarak görmüş oldum.

İş ilanlarını da BigQuery'ye taşıdım

BigQuery uygulamasını yalnızca hazır bir public dataset ile bırakmadım.

Türkiye ve global pazardan seçtiğim **20 Data odaklı iş ilanını** CSV formatında hazırlayıp kendi:

job_market_2026

tabloma yükledim.

Ardından SQL ile:

- en çok geçen teknik yetkinlikleri,
- Türkiye ve global ilanlar arasındaki farkları,
- hangi teknolojilerin birlikte öne çıktığını

analiz ettim.

Böylece çalışma şu akışa dönüştü:

İş İlanı → Veri Seti → BigQuery → SQL → Karşılaştırma → İçgörü

Bu yüzden BigQuery benim için bu ödevde yalnızca öğrenilecek başka bir cloud aracı olmadı.

Trend araştırmasını ölçülebilir hale getiren araç oldu.

Query completed

Using on-demand processing quota

Job information	Results	Visualization	JSON	Execution details	Execution graph
User	aslangamzenur079@gmail.com				
Location	US				
Creation time	Aug 30, 2026, 5:08:21 PM UTC+3				
Start time	Aug 30, 2026, 5:08:21 PM UTC+3				
End time	Aug 30, 2026, 5:08:22 PM UTC+3				
Duration	642 ms				
Bytes processed	1.15 GB				
Bytes billed	1.15 GB				
Slot milliseconds	8461				
Job priority	INTERACTIVE				
Use legacy SQL	false				
Destination table	Temporary table				
Labels					

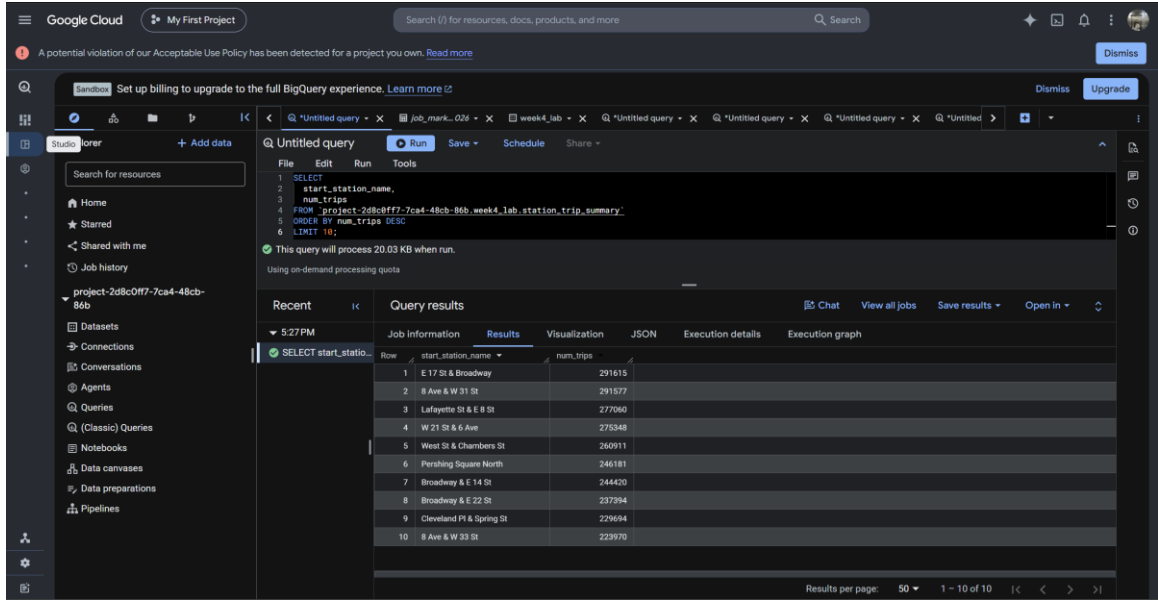
İlk sorguda işlenen veri miktarı: 1.15 GB

Query completed

Using on-demand processing quota

Job information	Results	Visualization	JSON	Execution details	Execution graph
Creation time	Aug 30, 2026, 5:12:19 PM UTC+3				
Start time	Aug 30, 2026, 5:12:19 PM UTC+3				
End time	Aug 30, 2026, 5:12:20 PM UTC+3				
Duration	346 ms				
Bytes processed	667.06 MB				
Bytes billed	668 MB				
Slot milliseconds	1584				
Job priority	INTERACTIVE				
Use legacy SQL	false				
Destination table	Temporary table				
Labels					

Yalnızca gerekli sütunlar seçildiğinde işlenen veri 667.06 MB'ye düştü



Önceden oluşturulan özet tablo üzerinden sorgulamada işlenen veri 20.03 KB'ye kadar düştü

3. Ontology for Web3 — Kimlikten “Anlam” Katmanına

“Ontology for Web3” kavramını araştırmaya başladığımda ilk karşıma çıkan şey beklediğimden farklıydı. Çünkü **Ontology** kelimesi burada iki ayrı kapı açıyor ve bunları birbirine karıştırmamak gerekiyor.

İlk kapı: Ontology bir Web3 projesi

Büyük harfle **Ontology**, kendisini Web3 için bir “trust layer” olarak konumlandıran blockchain ekosistemi. Özellikle dijital kimlik, itibar ve doğrulanabilir bilgiler üzerine yoğunlaşıyor. Ontology’nin **ONT ID** sistemi, W3C’nin Decentralized Identifier (DID) ve Verifiable Credential yaklaşımlarını temel alan merkeziyetsiz bir kimlik framework’ü. Amaç, kişinin kimliği ve verisi üzerindeki kontrolün tamamen merkezi platformlarda olmaması.

Buradaki problem aslında çok tanıdık:

Bugün internette;

**Google beni ayrı tanıyor,
üniversitem ayrı tanıyor,
LinkedIn ayrı tanıyor,
bankam ayrı tanıyor.**

Kimliğim farklı platformların veri tabanlarında parçalanmış durumda.

Web3’ün decentralized identity yaklaşımı ise farklı bir soru soruyor:

“Bir platform benim kim olduğumu söylemek yerine, sahip olduğum doğrulanabilir bir dijital kimliği farklı platformlara gösterebilseydim ne olurdu?”

Ontology'nin ONT ID yaklaşımında kullanıcıların kimlikleri ve ilişkili verileri üzerinde kontrol sahibi olması hedefleniyor; sistem decentralized identifiers ve verifiable credentials kullanıyor.

Fakat araştırmanın benim için daha ilginç kısmı bundan sonra başladı.

İkinci kapı: “ontology” aslında veriye anlam vermek

Bilgisayar bilimlerindeki **ontology** kavramı blockchain'den çok daha eski.

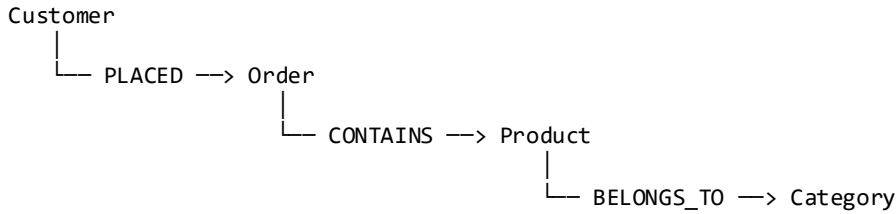
W3C'nin **OWL — Web Ontology Language** standardı, nesneler, nesne grupları ve bunların arasındaki ilişkiler hakkında karmaşık bilgilerin makine tarafından anlaşılabilir biçimde temsil edilmesi için tasarlanmış bir Semantic Web dili.

Örneğin normal bir veri tabanında şunlar olabilir:

```
customers
orders
products
tickets
```

Tabloların birbirlerine foreign key ile bağlı olduğunu biliyoruz.

Ontology yaklaşımında ise sadece bağlantıyı değil, **bağlantının ne anlama geldiğini** de ifade etmeye başlıyoruz:



Bu noktada elimizde artık sadece tablolar değil, **anlam taşıyan ilişkiler** oluşuyor.

Ve bu yapı Knowledge Graph yaklaşımına doğru ilerliyor.

Peki bunun AI ile ne ilgisi var?

Bence araştırmadaki asıl önemli soru burada ortaya çıktı:

Bir AI Agent'ın şirketteki tüm verilere erişebilmesi, o veriyi gerçekten anlayabildiği anlamına gelir mi?

Örneğin bir agent'a:

“Son 30 günde churn riski artan müşterileri bul ve bu müşterilerin yaşadığı sorunların hangi ürün kategorisiyle ilişkili olduğunu açıkla.”

dediğimizi düşünelim.

Agent'ın yalnızca tabloların isimlerini bilmesi yeterli olmayabilir.

Şunları da anlaması gerekir:

Customer nedir?

Order ile Customer nasıl ilişkilidir?

Ticket hangi müşteriye aittir?

Churn hangi metriği ifade ediyor?

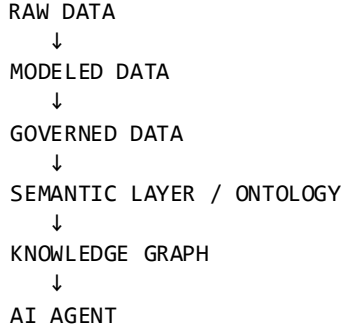
Product ve Category arasındaki ilişki nedir?

İşte **semantic layer, ontology ve knowledge graph** yaklaşımları burada anlam kazanmaya başlıyor.

2026'da bu ilişkinin yalnızca teorik olmadığını gösteren örnekler de bulunuyor. Örneğin Neo4j'nin güncel agent mimarilerinde semantic layer; agent'ın farklı veri tabanları, API'ler ve kurumsal sistemler arasında gezinirken entity, ontology, schema, lineage ve policy bilgilerini kullanabileceği bir "harita" olarak ele alınıyor.

Aynı şekilde 2026'da yayımlanan başka bir Neo4j çalışmasında ontology'nin bilgi grafinin yapısını ve ilişkilerin anlamını tanımladığı; knowledge graph'ın ise bu modeli gerçek verilerle dolduran katman olduğu anlatılıyor.

Burada benim gördüğüm dönüşüm şu:



İş ilanı analizimle bağlantısı

Bu araştırmanın benim için ilginç tarafı, daha önce BigQuery'de analiz ettiğim iş ilanlarıyla da birleşmesi oldu.

20 ilanlık örneklemimde:

**Data Modeling,
Governance / Quality,
dbt / Dataform**

gibi veri yapısını ve güvenilirliğini sağlayan yetkinlikler oldukça güçlü görünüyordu.

Ontology ve Semantic Layer ise henüz SQL veya Airflow gibi doğrudan her ilanda karşıma çıkan kavramlar değil.

Bu yüzden aralarında şöyle bir ayrım yapıyorum:

İş ilanları bana bugünün talebini gösteriyor.

Ontology + Knowledge Graph + Semantic Layer ise yarının veri altyapısına dair bir sinyal olabilir.

Bu kesin bir gelecek tahmini değil. Ancak AI Agent'ların kurumsal sistemlere daha fazla bağlandığı bir dünyada, agent'ların sadece **veriye erişmesi** değil, **verinin ne anlama geldiğini bilmesi** gerekecek.

Benim bu araştırmadan çıkardığım en güçlü cümle de bu oldu:

AI'ın sonraki problemi her zaman daha büyük bir model olmayabilir. Bazen asıl problem, modelin eriştiği verinin ne anlama geldiğinin tanımlanmamış olmasıdır.

Küçük ama önemli ayrım

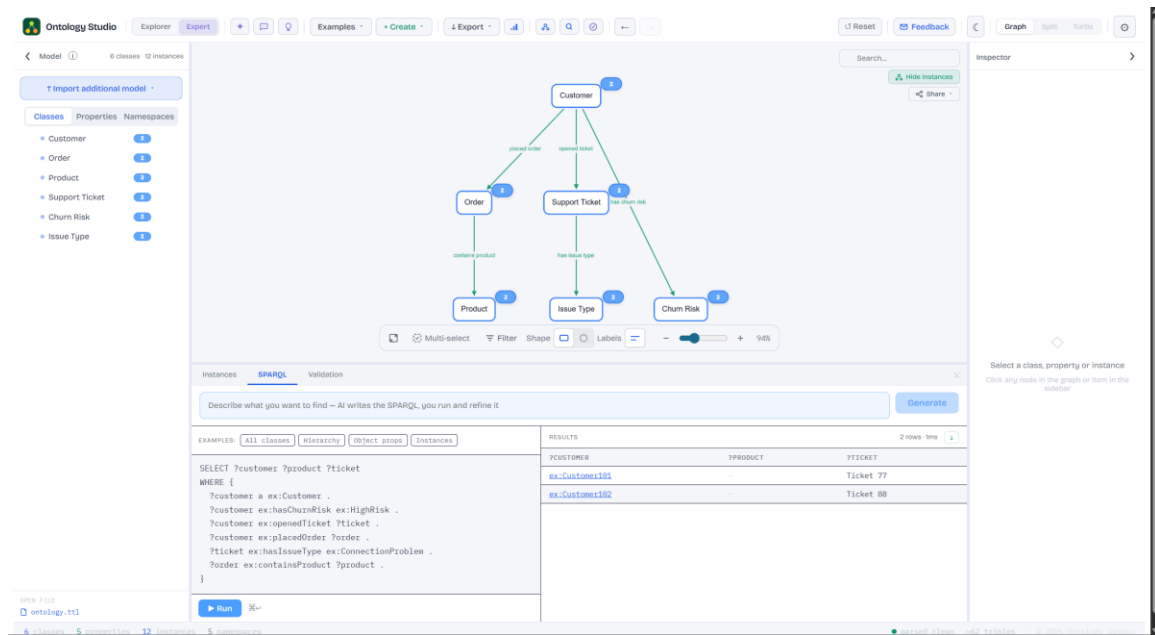
Bu araştırmada **Ontology blockchain projesi** ile bilgisayar bilimlerindeki **ontology kavramını aynı teknoloji olarak değerlendirmiyorum.**

Biri Web3 dünyasında decentralized identity ve trust altyapısı oluşturmayı hedefleyen bir proje; diğeri ise bilgiyi ve ilişkileri makine tarafından anlaşılabilir biçimde modellemeye yönelik daha genel bir bilgisayar bilimi kavramı.

Fakat ikisinin kesiştiği ilginç nokta:

identity + trust + verifiable information + machine-readable meaning

olduğu için Web3 ve AI'ın geleceği açısından birlikte incelenmeye değer olduklarını düşünüyorum.



Telecom domaini için oluşturduğum Ontology / Knowledge Graph modeli

The screenshot shows the RDF Playground interface. On the left, there is a list of RDF triples. On the right, a SPARQL query is entered and executed. The results are displayed in a table format.

```
2
3 ex:Customer101 a ex:Customer ;
4   ex:placedOrder ex:Order5001 ;
5   ex:openedTicket ex:Ticket77 ;
6   ex:hasChurnRisk ex:HighRisk .
7
8 ex:Order5001 a ex:Order ;
9   ex:containsProduct ex:RouterX .
10
11 ex:RouterX a ex:Product .
12
13 ex:Ticket77 a ex:SupportTicket ;
14   ex:hasIssueType ex:ConnectionProblem .
15
```

```
2
3 SELECT ?customer ?product ?ticket
4 WHERE {
5   ?customer a ex:Customer ;
6     ex:hasChurnRisk ex:HighRisk ;
7     ex:openedTicket ?ticket ;
8     ex:placedOrder ?order .
9
10   ?ticket ex:hasIssueType ex:ConnectionProblem .
11 }
```

Results

From: SPARQL

Result

```
1 customer,product,ticket
2 http://example.org/telecom/Customer101,http://example.org/telecom/RouterX,http://example.org/telecom/Ticket77
3
```

SPARQL sorgusu ile High Churn Risk + Connection Problem koşullarını sağlayan Customer101 → RouterX → Ticket77 ilişkisinin bulunması

4. Yetkinlik Geliştirme Planım — Teknoloji Biriktirmek Değil, Stack Kurmak

Trend araştırmasının sonunda önümde uzun bir teknoloji listesi oluştu. SQL, Python, BigQuery, Airflow, dbt/Dataform, Docker, Terraform, Data Quality, Knowledge Graph...

Fakat hepsini aynı anda öğrenmeye çalışmanın doğru olmadığını düşünüyorum.

Bu yüzden öğrenme planımı üç soruya göre oluşturdum:

İş ilanlarında ne kadar karşıma çıkıyor?

Öğrenmek istediğim diğer teknolojilerin temelini oluşturuyor mu?

Öğrendiğimi gösterebileceğim somut bir çıktı üretebilir miyim?

Bu bakış açısıyla benim için öğrenme yolu şu hale geldi:

Aşama 1 — Veriyi sorgulamaktan veriyi modellemeye

İlk odağım **SQL + Data Modeling** olacak.

BigQuery çalışmasında yalnızca sorgunun nasıl yazıldığının değil, verinin nasıl hazırlandığının da sonucu ciddi şekilde değiştirebildiğini gördüm.

Aynı analizde:

1.15 GB → 667.06 MB → 20.03 KB

şeklinde bir fark oluşması bana optimizasyonun yalnızca WHERE, LIMIT veya daha kısa SQL yazmak olmadığını gösterdi.

Bu nedenle SQL tarafında yalnızca sorgu yazmaya değil;

JOIN, CTE, Window Functions, Fact/Dimension, Star Schema, Grain ve Aggregation

konularını birlikte geliştirmek istiyorum.

Buradaki hedefim:

“SQL yazabiliyorum.” yerine “Analiz için doğru veri modelini tasarlayıp doğru sorguyu yazabiliyorum.” diyebilmek.

Aşama 2 — Analizi gerçek bir Data Pipeline'a dönüştürmek

İş ilanı analizimde en dikkat çekici sonuçlardan biri **Airflow ve dbt/Dataform** tarafındaydı.

Özellikle global örnekleme dbt/Dataform, Türkiye ve global örnekleme ise Airflow güçlü şekilde karşıma çıktı.

Bu nedenle BigQuery'den sonraki adımımı şöyle kuruyorum:

BigQuery → dbt/Dataform → Airflow

Buradaki mantık:

BigQuery

Verinin tutulduğu ve analiz edildiği yer.

↓

dbt / Dataform

Ham veriden temiz ve tekrar kullanılabilir analitik modeller üretme katmanı.

↓

Airflow

Bu işlemlerin hangi sırayla ve ne zaman çalışacağını yöneten orchestration katmanı.

Yani üç ayrı teknolojiyi öğrenmek yerine şu sistemi kurmayı hedefliyorum:

```
Raw Data
↓
BigQuery
↓
Staging
↓
dbt / Dataform
↓
Analytics Models
↓
Airflow
↓
Scheduled Pipeline
```

Bu aşamanın sonunda küçük ama gerçekten çalışan **uçtan uca bir data pipeline** oluşturmak istiyorum.

Aşama 3 — “Pipeline çalıştı” demek yeterli mi?

İş ilanı analizinde beni en fazla şaşırtan konulardan biri **Governance / Data Quality** oldu.

Bu noktada kendime şu soruyu sordum:

Pipeline her gün başarıyla çalışıyorsa ama yanlış veri üretiyorsa gerçekten başarılı sayılır mı?

Cevabım hayır.

Bu nedenle sonraki aşamada projelerime:

NULL kontrolü
duplicate kontrolü
primary key testi
accepted values
freshness
schema kontrolü
lineage

gibi kontroller eklemek istiyorum.

Böylece hedefim yalnızca veri üretmek değil:

Güvenilebilir veri üretmek.

Bu bölüm özellikle önemli çünkü yaptığım ilan analizinde Data Quality/Governance'ın yalnızca teorik bir konu olmadığını, doğrudan iş ilanlarında karşılık bulduğunu gördüm.

Aşama 4 — Bugünün stack'inden yarının veri katmanına

Ontology araştırması öğrenme planıma farklı bir alan ekledi.

Canlı uygulamada:

Customer → Order → Product
Customer → Ticket → Issue Type
Customer → Churn Risk

ilişkilerini bir Knowledge Graph olarak modelledim.

Daha sonra SPARQL ile:

“Yüksek churn riskine sahip ve bağlantı problemi yaşayan müşterinin kullandığı ürün hangisi?”

sorusunu sorguladım ve:

Customer101 → RouterX → Ticket77

sonucuna ulaştım.

Bu deneyden sonra **Semantic Layer, Knowledge Graph ve Ontology** konularını şu anda SQL veya Airflow kadar öncelikli görmüyorum; fakat AI Agent sistemlerinin gelişimi açısından takip etmek istediğim bir alan olarak görüyorum.

Bu yüzden öğrenme planımda küçük bir “**Frontier Track**” olacak:

Semantic Layer → Knowledge Graph → Ontology → AI Agents

Burada amaç hemen uzmanlaşmak değil.

Amacım:

Bugünün teknolojilerinde güçlü olurken, yarının veri mimarisinin hangi yöne gittiğini de kaçırmamak.

Benim Öğrenme Rotam

AŞAMA	ODAK	SOMUT ÇIKTI
1	SQL + Data Modeling	Star Schema + analitik SQL
2	BigQuery	Gerçek veri üzerinde cloud analytics
3	dbt / Dataform	Raw → Staging → Mart modeli
4	Airflow	Otomatik çalışan DAG
5	Data Quality & Governance	Test ve validation katmanı
6	Ontology & Knowledge Graph	RDF + SPARQL mini uygulaması
7	AI Agents	Semantic data kullanan agent deneyi

Ancak benim için artık bir konuyu “öğrenmek”, kursu tamamlamak anlamına gelmiyor.

Yeni öğrenme döngümü şöyle tanımlıyorum:

Araştır → Kur → Boz → Ölç → Açıkla

BigQuery’de sorguları değiştirip işlenen veriyi ölçmem veya Ontology tarafında modeli gerçekten oluşturup SPARQL sorgusu çalıştırmam bu yöntemin ilk örnekleri oldu.

Bu nedenle bundan sonraki hedefim mümkün olduğunca fazla teknoloji öğrenmek değil.

Bir veri problemini uçtan uca çözebilecek doğru teknoloji zincirini kurabilmek ve öğrendiğim her yetkinlik için görülebilir bir kanıt üretebilmek.

5. Görünürlük Stratejim — Paylaşmak Değil, Deneyimletmek

Bu çalışmada görünürlük stratejimi yalnızca “LinkedIn’de paylaşım yapmak” üzerine kurmak istemedim. Çünkü teknik bir çalışmayı anlatmanın en güçlü yollarından birinin, insanların o çalışmayı **kendilerinin inceleyebilmesi ve deneyebilmesi** olduğunu düşünüyorum.

Bu nedenle merkezde **Hugging Face** olacak.

Hugging Face Dataset — Araştırmanın Verisini Açmak

Trend araştırmasında kullandığım 20 iş ilanını yalnızca kendi bilgisayarımda tutmak yerine, oluşturduğum `job_market_2026` veri setini Hugging Face üzerinde yayınlamayı planlıyorum.

Veri setinde ilanların Türkiye / Global ayrımıyla birlikte SQL, Python, BigQuery, Airflow, dbt/Dataform, Governance, Data Modeling gibi yetkinliklerin ilanlarda açıkça bulunup bulunmadığını gösteren alanlar yer alacak.

Böylece araştırmanın arkasındaki veri de görülebilir olacak.

Benim için burada önemli olan:

“Bu sonuçlara ulaştım.” demek yerine, “Kullandığım veri burada; istersen sen de inceleyebilirsin.” diyebilmek.

Dataset Card bölümünde de veriyi nasıl oluşturduğumu, 1/0 değerlerinin ne anlama geldiğini ve 20 ilanlık örneklemin piyasanın tamamını temsil etmediğini açıkça belirtmek istiyorum.

Hugging Face Space — Araştırmayı Küçük Bir Ürüne Dönüştürmek

Görünürlük stratejisinin asıl farklı kısmını burada oluşturmak istiyorum.

İş ilanı analizini yalnızca tablo veya ekran görüntüsü olarak paylaşmak yerine küçük bir **interaktif Data Trends Space** hazırlamayı planlıyorum.

Space açıldığında kullanıcı örneğin:

Türkiye | Global | Tümü

seçeneklerinden birini seçebilecek.

Seçime göre SQL, Python, Airflow, dbt/Dataform, BigQuery, Governance gibi yetkinliklerin görülme oranları değişecek.

Ayrıca ayrı bir bölümde:

Türkiye vs Global Skill Gap

karşılaştırması gösterilecek.

Böylece kullanıcı benim yazdığım sonuçları yalnızca okumayacak; veriyi kendisi keşfedebilecek.

Space içerisinde ikinci küçük alanı ise **Future Signals** olarak tasarlamak istiyorum.

Burada bu hafta araştırdığım:

Ontology → Knowledge Graph → Semantic Layer → AI Agents

bağlantısını gösterebilirim.

Hatta canlı uygulamada oluşturduğum telecom örneğini küçük bir demo halinde sunabilirim:

High Churn Risk + Connection Problem

sorgusunun sonucunda:

Customer101 → RouterX → Ticket77

ilişkisinin bulunması gibi.

Böylece tek Space içerisinde hem **bugünün iş piyasasından ölçtüğüm sinyaller**, hem de **gelecekte takip etmek istediğim teknolojiler** bulunmuş olacak.

GitHub — Nasıl Yaptığımı Göstermek

Hugging Face sonucu gösterirken GitHub projenin teknik tarafını tutacak.

Burada BigQuery sorgularımı, kullandığım CSV veri setini, RDF/Turtle ontology modelini, SPARQL sorgusunu ve Space'in kaynak kodunu paylaşmak istiyorum.

Yani platformların görevleri birbirinden farklı olacak:

Hugging Face Dataset → Veriyi göster

Hugging Face Space → Deneyimlet

GitHub → Nasıl yaptığımı göster

LinkedIn → İnsanları projeye getir

LinkedIn — Sonucu Değil, Merakı Paylaşmak

LinkedIn paylaşımında bütün araştırmayı uzun uzun anlatmak yerine merak uyandıran birkaç bulgu kullanmayı tercih ederim.

Örneğin:

Bu hafta “2026'da hangi teknolojiler trend?” sorusuna hazır bir listeyle cevap vermek istemedim.

Bunun yerine 20 güncel Data ilanını küçük bir veri setine dönüştürdüm ve BigQuery üzerinde analiz ettim.

Sonuçlarda SQL ve Python zaten güçlüydü; fakat Airflow, dbt/Dataform ve Data Governance tarafındaki yoğunluk benim için daha dikkat çekiciydi.

Araştırmanın sonunda veriyi sadece raporlamak yerine Hugging Face üzerinde insanların kendilerinin keşfedebileceği küçük bir interaktif çalışma haline getirmeye karar verdim.

Devamında **Hugging Face Space** ve **GitHub** bağlantısını paylaşabilirim.

Benim Görünürlük Yaklaşımım

Bu çalışmadan sonra görünürlüğü yalnızca “ne kadar paylaşım yaptığım” olarak değerlendirmiyorum.

Benim için daha anlamlı olan zincir:

Araştır → Veri üret → Analiz et → Ürüne dönüştür → İnsanların deneyimlemesine izin ver.

Bu nedenle bu haftaki çalışmanın son çıktısı yalnızca bir ödev dosyası olmayacak.

İncelenebilir bir dataset, çalışan bir demo ve teknik olarak doğrulanabilir bir proje bırakmak istiyorum.

Uygulama — Araştırmayı Canlı Bir Ürüne Dönüştürmek

Görünürlük stratejisini yalnızca planlamak yerine bu hafta doğrudan uyguladım. İş ilanlarından oluşturduğum `job_market_2026` veri setini **Hugging Face Dataset** olarak yayınladım ve metodolojisini bir Dataset Card ile dokümante ettim.

Ardından aynı veriyi kullanan interaktif bir **Hugging Face Space** geliştirdim. Space üzerinde kullanıcılar Türkiye, Global veya tüm ilanları seçerek teknik yetkinliklerin görülme oranlarını inceleyebiliyor ve iki pazar arasındaki skill gap'leri karşılaştırabiliyor.

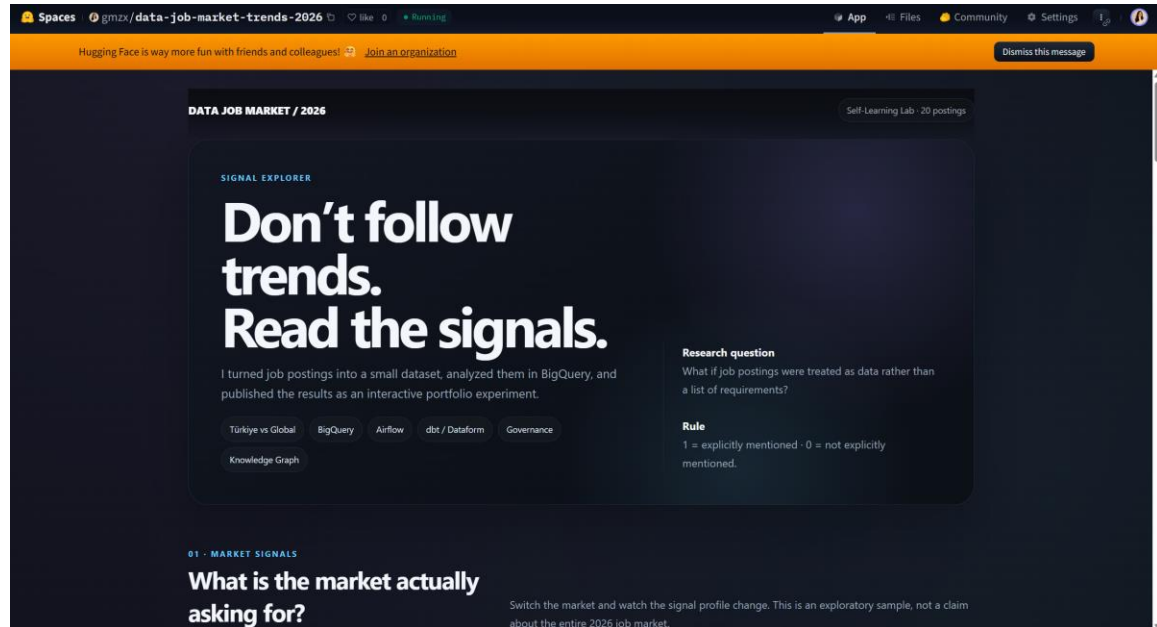
Çalışmanın **Future Signals** bölümünde ise Ontology araştırmamı küçük bir semantic-data demosuna dönüştürdüm. Kullanıcı churn riskini ve problem türünü seçtiğinde sistem ilgili müşteri-ürün-ticket ilişkisini gösteriyor.

Böylece bu haftaki çalışma:

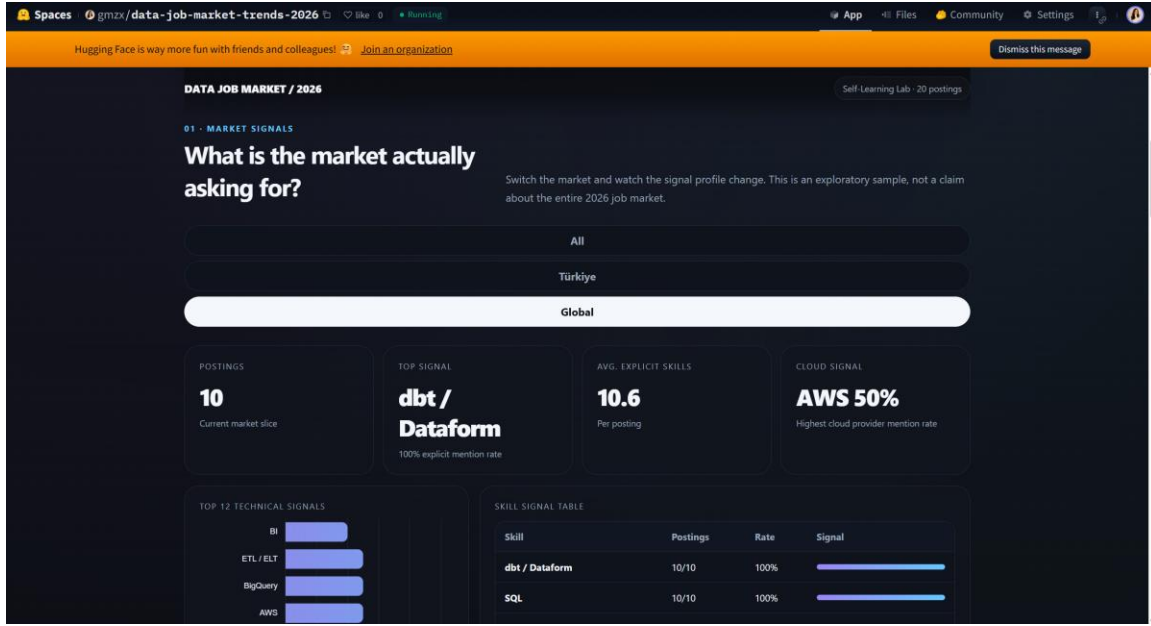
Araştırma → Dataset → BigQuery Analizi → Ontology Deneyi → Canlı Demo

şeklinde uçtan uca bir çıktıya dönüştü.

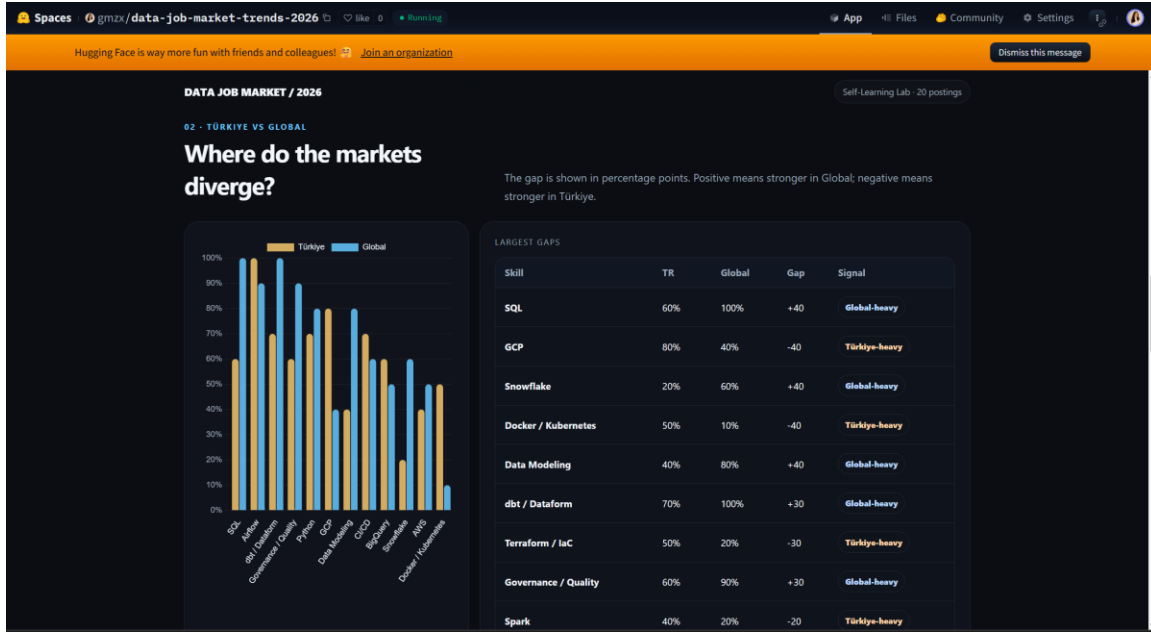
Bu çalışma sonunda görünürlüğü yalnızca yaptığım şeyi paylaşmak değil, başkalarının yaptığım şeyi deneyebilmesini sağlamak olarak görmeye başladım.



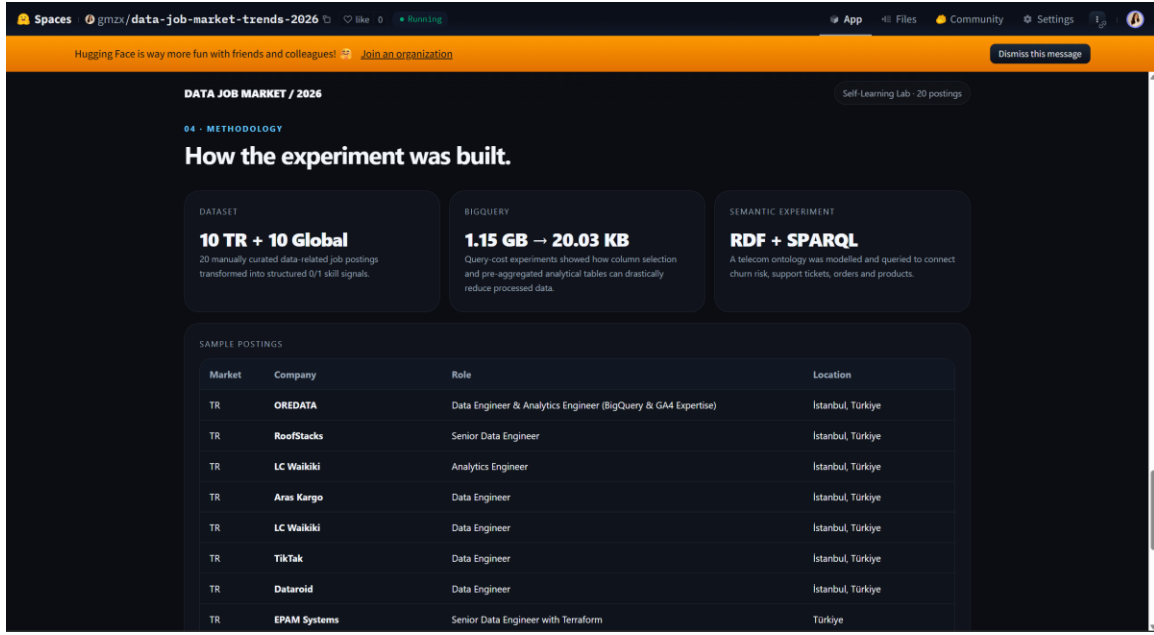
İş ilanlarından oluşturduğum veri setini Hugging Face üzerinde yayınladım.



İş ilanı sinyallerini interaktif olarak keşfetmek için oluşturduğum Hugging Face Space.



Türkiye ve global örneklem arasındaki teknik yetkinlik farklarının karşılaştırılması.



Ontology araştırmasını Space içinde etkileşimli semantic-data demosuna dönüştürdüm.

6. Sonuç — Trend Takibinden Kendi Öğrenme Sistemime

Bu çalışmaya başlarken amacım güncel teknoloji trendlerini araştırmaktı. Fakat süreç ilerledikçe yalnızca trendleri listelemek yerine, onları **ölçmeye, denemeye ve görünür hale getirmeye** başladım.

İş ilanlarını veri setine dönüştürerek hangi yetkinliklerin gerçekten tekrar ettiğini analiz ettim. BigQuery üzerinde sorgu maliyetlerini karşılaştırdım ve veri modellemenin performans etkisini uygulamalı olarak gördüm. Ontology ve Knowledge Graph tarafında ise kavramı teoride bırakmayıp RDF/Turtle ve SPARQL ile küçük bir semantic-data deneyi gerçekleştirdim.

Son aşamada tüm bu çalışmayı Hugging Face üzerinde yayınlanan bir dataset ve interaktif bir Space'e dönüştürdüm.

Çalışmanın sonunda benim için en önemli değişim şu oldu:

Trendleri takip etmek yerine, sinyalleri okuyup onları kendi öğrenme planıma dönüştürmek.

Artık yeni bir teknolojiyle karşılaştığımda yalnızca “Bunu öğrenmeli miyim?” diye sormak istemiyorum.

Bunun yerine:

Neden kullanılıyor?

İş dünyasında gerçekten karşılığı var mı?

Hangi teknolojilerle birlikte çalışıyor?

Küçük bir uygulamayla test edebilir miyim?

Öğrendiğimi görünür bir çıktıya dönüştürebilir miyim?

sorularını sormak istiyorum.

Bu nedenle benim için Self-Learning artık:

Araştır → Veriyle doğrula → Uygula → Ölç → Ürüne dönüştür → Paylaş

şeklinde ilerleyen bir süreç.

Ve bu haftadan geriye yalnızca bir ödev değil;

bir BigQuery deneyi, bir iş piyasası veri seti, bir Knowledge Graph uygulaması ve insanların kendilerinin deneyebileceği canlı bir proje

kalmış oldu.

Belki de asıl yetkinlik, her yeni teknolojiyi bilmek değil; hangisinin gerçekten öğrenmeye değer olduğunu anlayabilmek.

